

Simulación de Entrevista Técnica — Apache Kafka

Guía de estudio en formato entrevista · topics, particiones, consumer groups, garantías de entrega y KRaft en el sistema POS

Guía de estudio en formato entrevista sobre **Apache Kafka**, el bus de eventos que conecta los servicios del POS y sostiene el flujo SAGA. Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y **follow-ups**.

Formato sugerido: 50–60 min. Bloques: fundamentos (10'), particiones/consumer groups (15'), garantías de entrega (15'), en el proyecto (10'), operación (10').

0. Warm-up — Kafka en el proyecto

P ¿Qué papel juega Kafka en este sistema POS?

R Kafka (:9092) es el **bus de eventos asíncrono** entre servicios y el sustrato del **flujo SAGA** de una venta. `venta-service` (orquestador) y `stock / despacho` (participantes) se comunican por **topics**: `stock-reserve-topic` , `stock-reserve-response-topic` , `despacho-request-topic` , `despacho-response-topic` y `stock-compensate-topic` . Kafka **desacopla** a los servicios y da tolerancia a picos y caídas temporales (los mensajes quedan persistidos en el topic).

1. Fundamentos

P1 ¿Qué es Kafka y en qué se diferencia de una cola tradicional?

R Kafka es un **log distribuido de commits**, no una cola clásica. Los mensajes se **persisten** en un log ordenado e inmutable y **no se borran al leerlos** (se retienen por tiempo/tamaño). Varios consumidores pueden leer el mismo topic de forma independiente, cada uno con su **offset**. Una cola tradicional (RabbitMQ) suele borrar el mensaje tras el ack. Kafka prioriza alto throughput, reproducibilidad (re-lectura) y desacople productor/consumidor.

P2 Define topic, partición y offset.

R Un **topic** es un canal lógico de mensajes. Se divide en **particiones** (logs paralelos) que dan escalabilidad y paralelismo. Cada mensaje dentro de una partición tiene un **offset** secuencial que identifica su posición. El consumidor guarda hasta qué offset ha procesado (*commit*), lo que le permite reanudar tras un reinicio.

2. Particiones y consumer groups

P3 ¿Cómo funcionan los consumer groups y cómo escalan el consumo?

R Un **consumer group** reparte las particiones de un topic entre sus consumidores: cada partición la lee **un solo** consumidor del grupo a la vez. Para escalar se añaden consumidores hasta el número de particiones (más consumidores que particiones → algunos ociosos). En el proyecto cada servicio tiene su grupo (`venta-group` , `stock-group` , `despacho-group`), y grupos distintos **reciben todos** los mensajes de forma independiente.

FOLLOW-UP ¿Qué es un rebalanceo y por qué duele? — Cuando entra/sale un consumidor, Kafka **reassigna** particiones. Durante el rebalanceo el consumo se pausa, y tras él un consumidor puede reprocesar mensajes (de ahí la necesidad de **idempotencia**).

P4 ¿Cómo se elige la partición de un mensaje? ¿Por qué particionar por `orderId` ?

R Si el mensaje tiene **clave**, Kafka aplica `hash(key) % numParticiones` ; sin clave, round-robin. Particionar por **orderId** garantiza que todos los eventos de una misma orden caen en la **misma partición** y por tanto se procesan **en orden**, que es exactamente lo que necesita la SAGA (reservar → despachar → completar/compensar en secuencia).

3. Garantías de entrega y orden

P5 Explica at-most-once, at-least-once y exactly-once.

R

- **At-most-once**: commit del offset *antes* de procesar → si falla, se pierde el mensaje (0 o 1 vez).
- **At-least-once** (por defecto): procesar y *luego* commitear → si falla entre medias, se reprocesa (1 o más veces). Requiere **idempotencia**.
- **Exactly-once** (EOS): con productor idempotente + transacciones de Kafka; el efecto es "una sola vez" dentro de Kafka, a costa de complejidad y rendimiento.

Este proyecto asume **at-least-once** y resuelve los duplicados con idempotencia por `orderId`.

P6 ¿Cómo garantizas que no se pierden mensajes del lado productor?

R

Con `acks=all` (el líder espera el ack de las réplicas **in-sync**) + réplicas (`replication.factor ≥ 3`) + `min.insync.replicas ≥ 2`. Así, aunque caiga un broker, el mensaje ya está replicado. Con `acks=1` se puede perder si el líder cae antes de replicar; con `acks=0` se maximiza throughput pero sin garantías.

FOLLOW-UP ¿Qué es el ISR (in-sync replicas)? — El conjunto de réplicas al día con el líder. Si una se atrasa, sale del ISR; el líder solo confirma `acks=all` contra el ISR.

P7 ¿Kafka garantiza orden global?

R

No: solo garantiza orden **dentro de una partición**, no entre particiones de un topic. Por eso el orden se diseña eligiendo la **clave de particionado** (aquí `orderId`): lo que debe ir ordenado comparte partición.

4. Kafka en el flujo SAGA

P8 Recorre los topics de la SAGA y quién produce/consume.

R

Topic	Produce	Consume
stock-reserve-topic	venta	stock
stock-reserve-response-topic	stock	venta
despacho-request-topic	venta	despacho
despacho-response-topic	despacho	venta
stock-compensate-topic	venta	stock

El patrón es **request/response por eventos**: venta emite comandos y reacciona a las respuestas para avanzar la máquina de estados de la orden.

P9 Relación entre Kafka, el problema dual-write y Outbox.

R Guardar la orden en Mongo y publicar en Kafka son **dos escrituras no atómicas**: si el servicio cae entre ambas, la SAGA se cuelga. El patrón **Transactional Outbox** resuelve esto escribiendo el evento en una tabla outbox **dentro de la misma transacción** que el cambio de estado, y un relay (polling o Debezium/CDC) lo publica a Kafka después. La publicación sigue siendo at-least-once, de ahí la idempotencia en el consumidor.

5. Operación: KRaft, retención, DLQ

P10 ¿Qué es KRaft y por qué se usa en K8s?

R **KRaft** (Kafka Raft) elimina la dependencia de **ZooKeeper**: Kafka gestiona sus propios metadatos con un quórum de controladores usando Raft. Ventaja operativa: **menos componentes** que desplegar y monitorear (clave en Kubernetes, donde el manifiesto `03-kafka.yaml` corre Kafka en modo KRaft), arranque más simple y mejor escalabilidad de metadatos.

P11 ¿Qué es el consumer lag y por qué monitorearlo?

R El **lag** es la diferencia entre el último offset producido y el último commiteado por el consumer group: cuántos mensajes le faltan por procesar. Un lag creciente indica que el consumidor no da abasto (o está caído). En la SAGA, vigilar el lag de `despacho-group` avisa si los despachos se están atrasando; se escala añadiendo consumidores/particiones.

FOLLOW-UP ¿Y un DLQ? — Un **dead-letter topic** recibe los mensajes que fallan repetidamente tras reintentos con backoff, para no bloquear la partición y permitir inspección/reproceso manual.

6. Diseño abierto / escenarios

P12 Un consumidor procesa un evento dos veces. ¿Qué haces?

R Es esperable con at-least-once (p. ej. tras un rebalanceo o un fallo antes del commit). Se mitiga con **idempotencia**: claves por `orderId`, operaciones **upsert**, verificar el estado actual antes de aplicar (no reservar si ya está reservado) y **deduplicar** por id de evento. Las compensaciones también deben ser idempotentes.

P13 Llega Black Friday y sube el tráfico de ventas. ¿Cómo escalas Kafka?

R Aumentar el **número de particiones** de los topics calientes (permite más consumidores en paralelo por grupo) y añadir **instancias** de los servicios consumidores dentro de su group. Kafka actúa como **buffer** natural que absorbe el pico mientras los consumidores procesan a su ritmo. Cuidar la clave de particionado para no crear *hot partitions* y monitorear el lag.

Apéndice — Chuleta rápida

Tema	Punto clave
Modelo	Log distribuido de commits; mensajes persistidos, no se borran al leer
Topic/partición/offset	Canal → particiones (paralelismo) → offset (posición por partición)
Consumer group	1 partición → 1 consumidor del grupo; escalar hasta nº de particiones
Particionado	<code>hash(orderId)</code> → orden garantizado por orden dentro de su partición
Entrega	at-least-once (default) → idempotencia por <code>orderId</code>
Durabilidad	<code>acks=all</code> + <code>replication.factor ≥ 3</code> + <code>min.insync.replicas ≥ 2</code>
Orden	Solo dentro de una partición, no global
Topics SAGA	<code>reserve</code> / <code>reserve-response</code> / <code>despacho-request</code> / <code>despacho-response</code> / <code>compensate</code>
KRaft	Sin ZooKeeper; menos componentes (usado en K8s)
Operación	Monitorear consumer lag; DLQ + reintentos con backoff